Operating Systems: Internals and Design Principles

Chapter 4 Threads

Seventh Edition

By William Stallings

Operating Systems: Internals and Design Principles

The basic idea is that the several components in any complex system will perform particular subfunctions that contribute to the overall function.

—THE SCIENCES OF THE ARTIFICIAL,

Herbert Simon

Processes and Threads

Traditional processes have two characteristics:

Resource Ownership

Process includes a virtual address space to hold the process image

- the OS provides protection to prevent unwanted interference between processes with respect to resources

Scheduling/Execution

Follows an execution path that may be interleaved with other processes

- a process has an execution state (Running, Ready, etc.) and a dispatching priority and is scheduled and dispatched by the OS
- Traditional processes are *sequential*; i.e. only *one* execution path

Processes and Threads

- ***Multithreading*** **-** The ability of an OS to support multiple, concurrent paths of execution within a single process
- The unit of resource ownership is referred to as a ***process*** or ***task***
- The unit of dispatching is referred to as a ***thread*** or ***lightweight process***

Single Threaded Approaches

- A single execution path per process, in which the concept of a thread is not recognized, is referred to as a single-threaded approach
- MS-DOS, some versions of UNIX supported only this type of process.

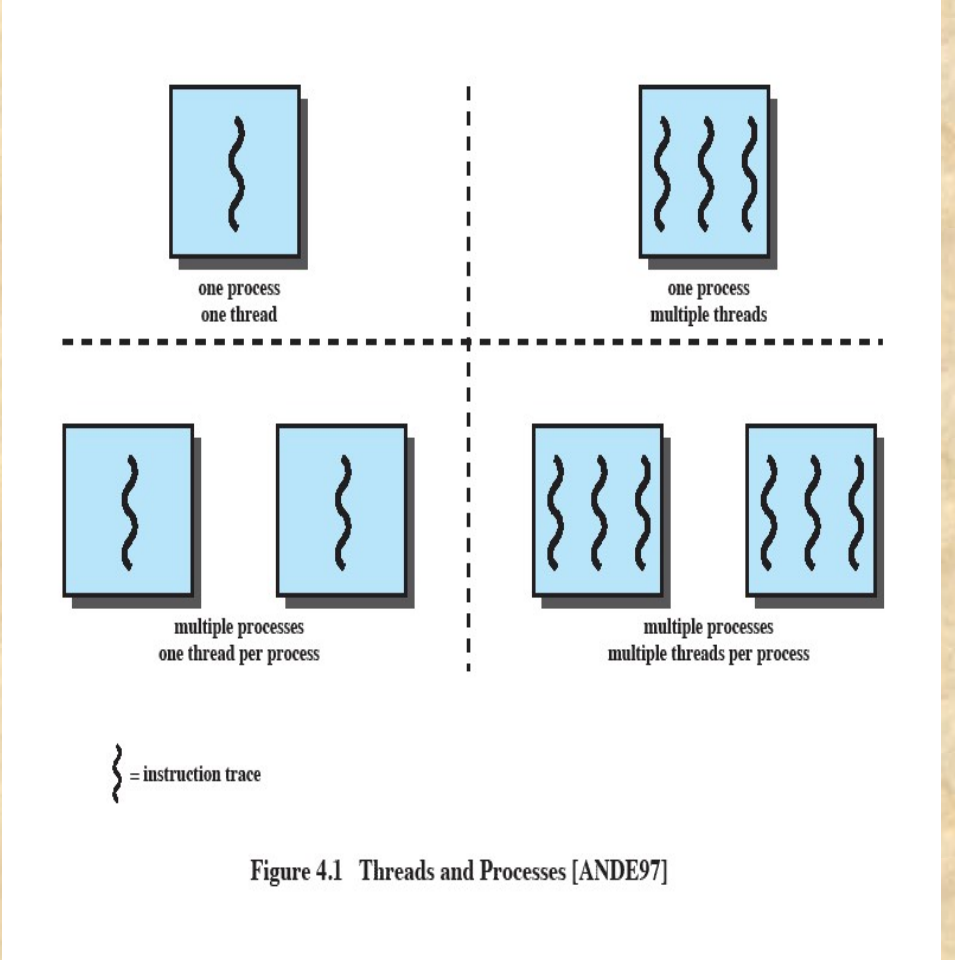


Figure 4.1 Threads and Processes [ANDE97]

Multithreaded Approaches

- The right half of Figure 4.1 depicts multithreaded approaches
- A Java run-time environment is a system of *one* process with multiple threads; Windows, some UNIXes, support *multiple* multithreaded processes.

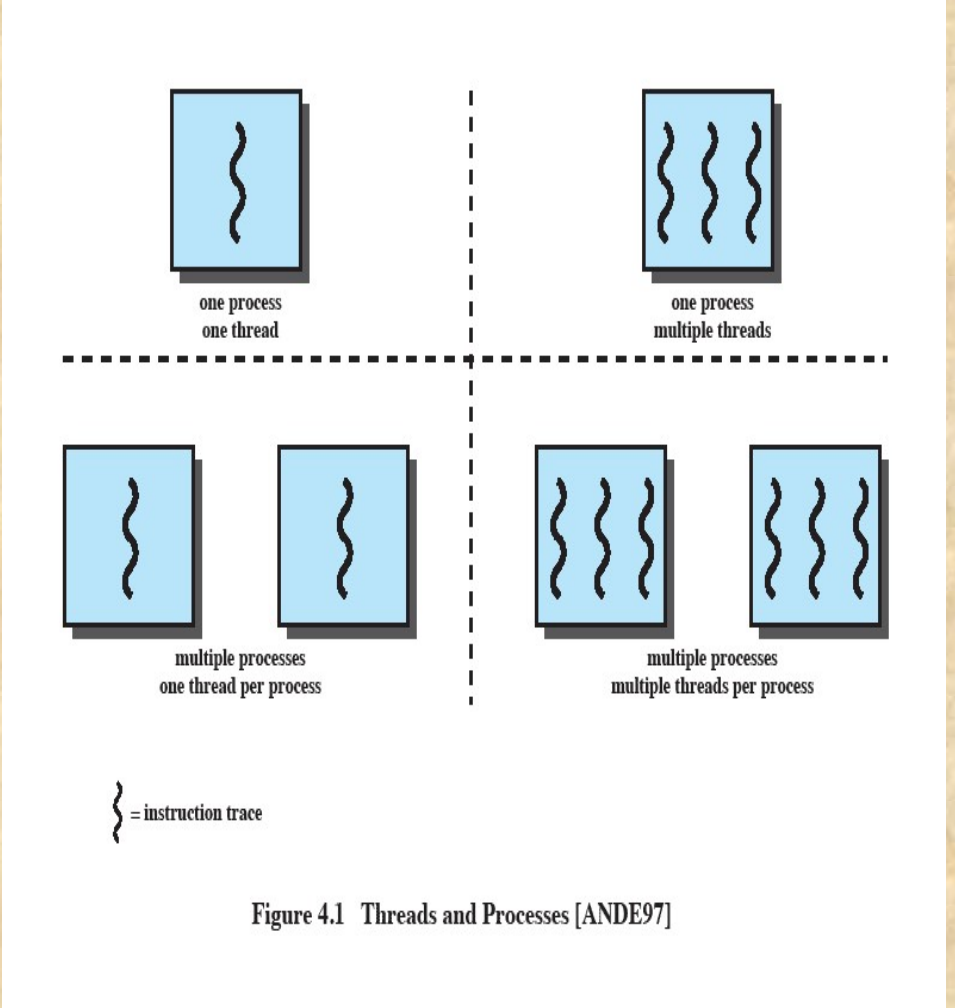


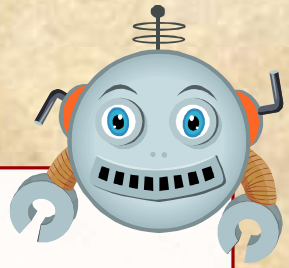
Figure 4.1 Threads and Processes [ANDE97]

Processes

- In a multithreaded environment the process is the unit that owns resources and the unit of protection.
  - i.e., the OS provides protection at the process level
- Processes have
  - A virtual address space that holds the process image
  - Protected access to
    - processors
    - other processes
    - files
    - I/O resources

One or More Threads in a Process

Each thread has:



- an execution state (Running, Ready, etc.)
- saved thread context when not running (TCB)
- an execution stack
- some per-thread static storage for local variables
- access to the shared memory and resources of its process (all threads of a process share this)

Threads vs. Processes

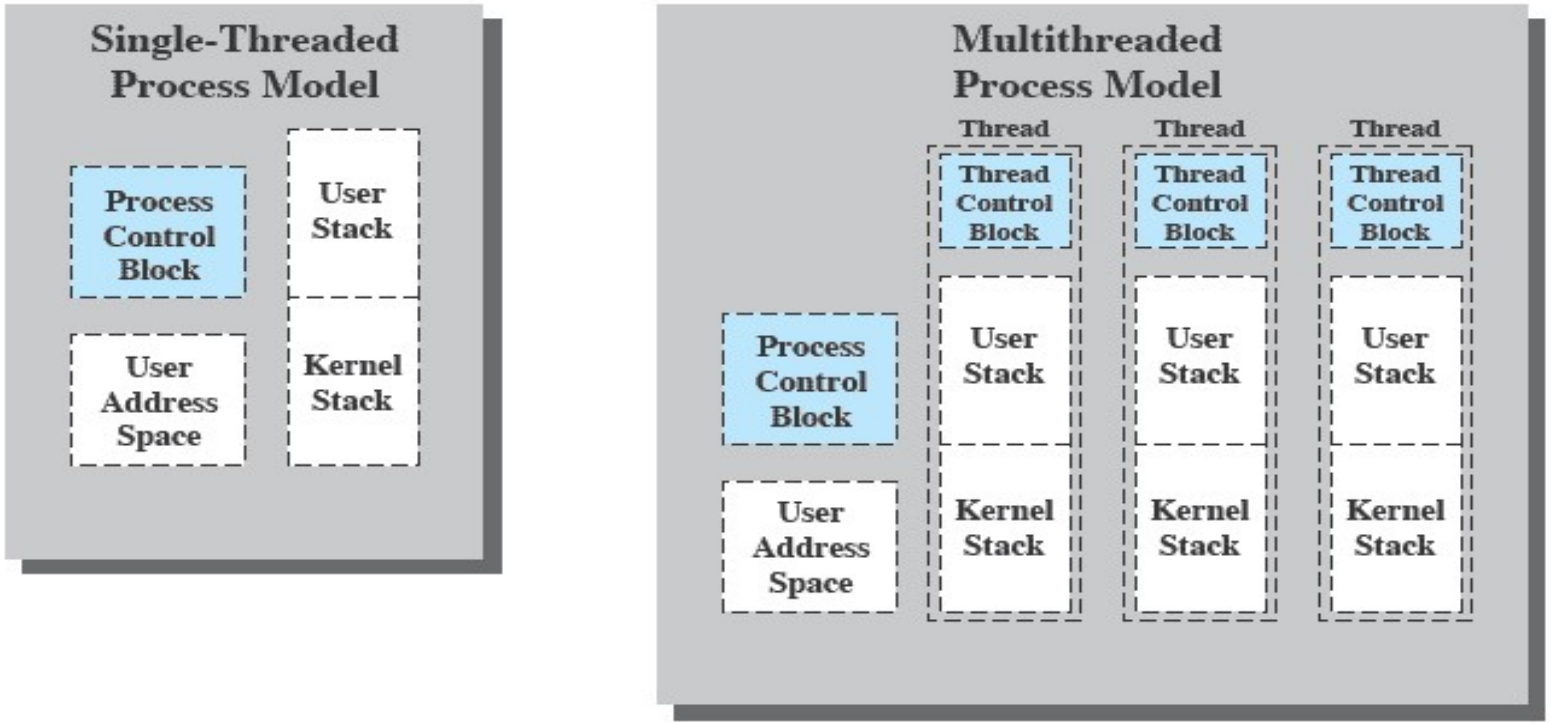
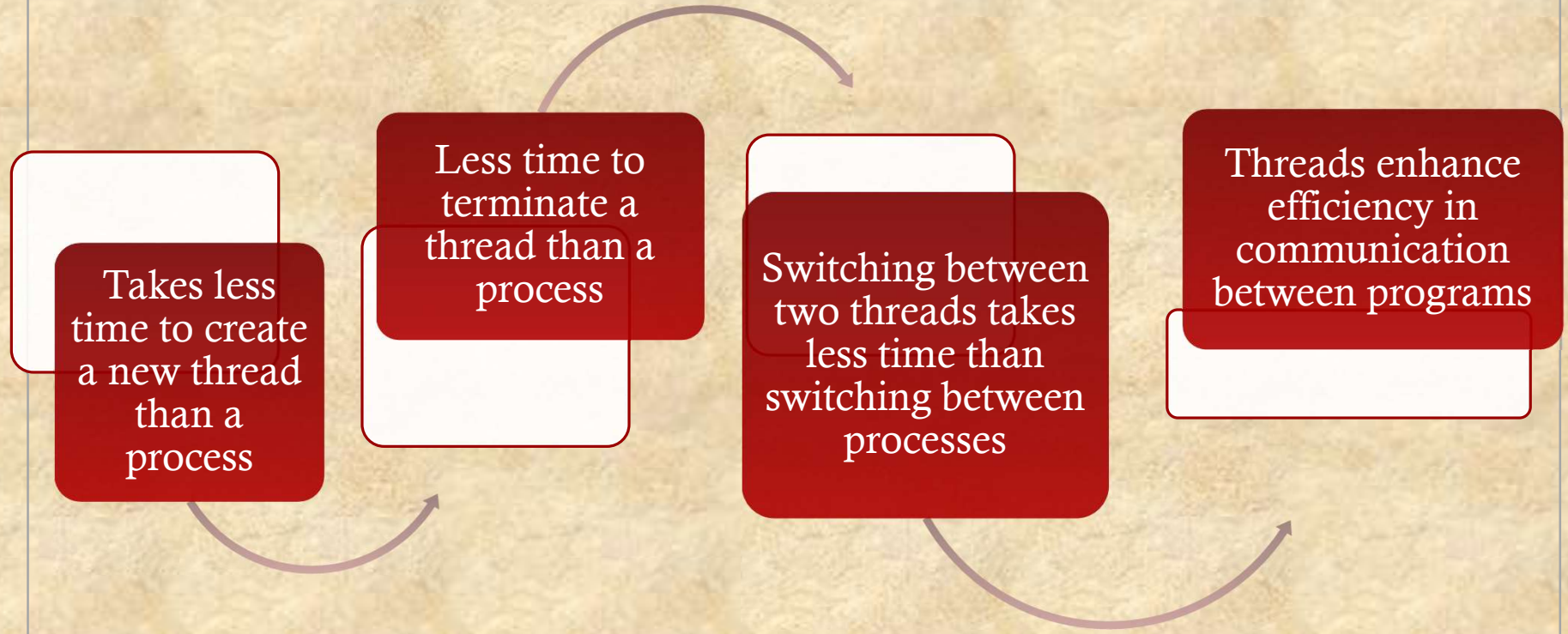
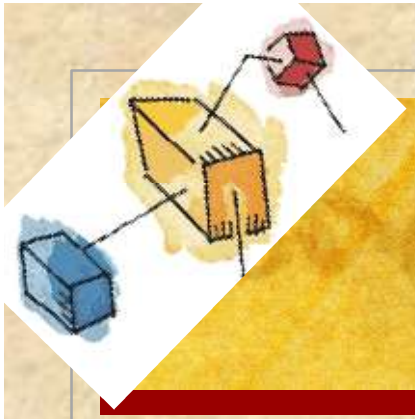


Figure 4.2 Single Threaded and Multithreaded Process Models

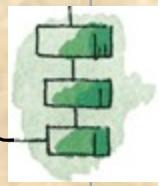
Benefits of Threads





Benefits

- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of process, easier than shared memory or message passing
- **Economy –** cheaper than process creation, thread switching lower overhead than context switching
- **Scalability –** process can take advantage of multiprocessor architectures



Thread Use in a Single-User System

- Foreground and background work
- Asynchronous processing
- Speed of execution
- Modular program structure

Threads

- In an OS that supports threads, scheduling and dispatching is done on a thread basis

Most of the state information dealing with execution is maintained in thread-level data structures

- suspending a process involves suspending all threads of the process
- termination of a process terminates all th within the process

Thread Execution States

The key states for a thread are:

- Running
- Ready
- Blocked

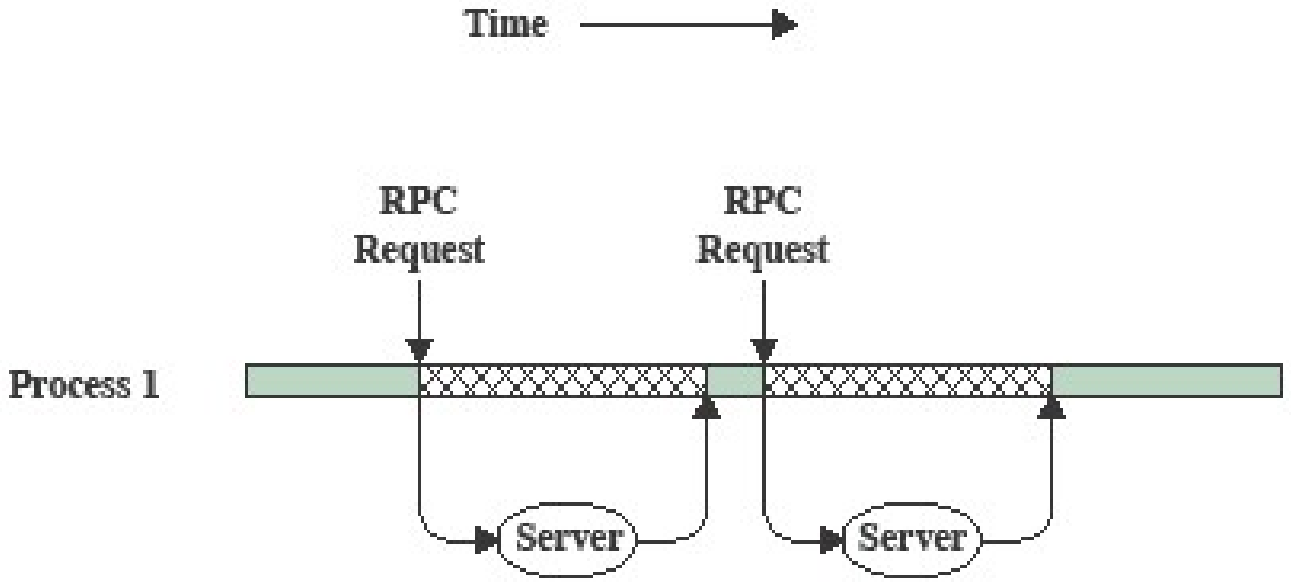
Thread operations associated with a change in thread state are:

- Spawn (create)
- Block
- Unblock
- Finish

Thread Execution

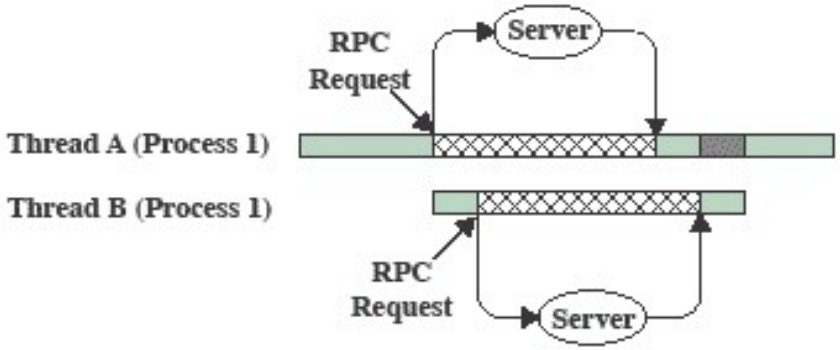
- A key issue with threads is whether or not they can be scheduled independently of the process to which they belong.
- Or, is it possible to block one thread in a process without blocking the entire process?
  - If not, then much of the flexibility of threads is lost.

RPC Using Single Thread



(a) RPC Using Single Thread

RPC Using One Thread per Server



(b) RPC Using One Thread per Server (on a uniprocessor)

Blocked, waiting for response to RPC

Blocked, waiting for processor, which is in use by Thread B

Running

Multithreading on a Uniprocessor

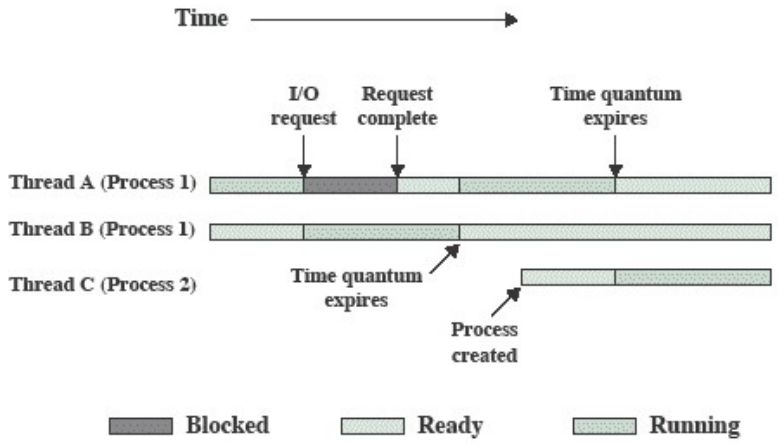
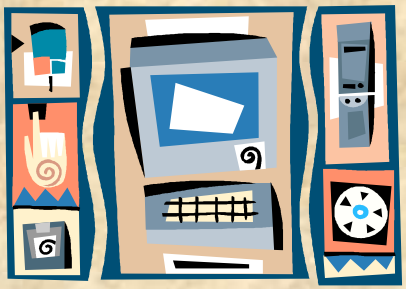


Figure 4.4 Multithreading Example on a Uniprocessor



Thread Synchronization

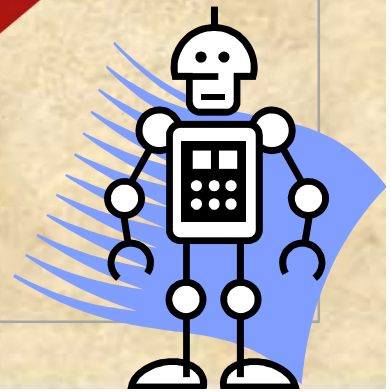
- It is necessary to synchronize the activities of the various threads
  - all threads of a process share the same address space and other resources
  - any alteration of a resource by one thread affects the other threads in the same process

Types of Threads

User Level Thread (ULT)

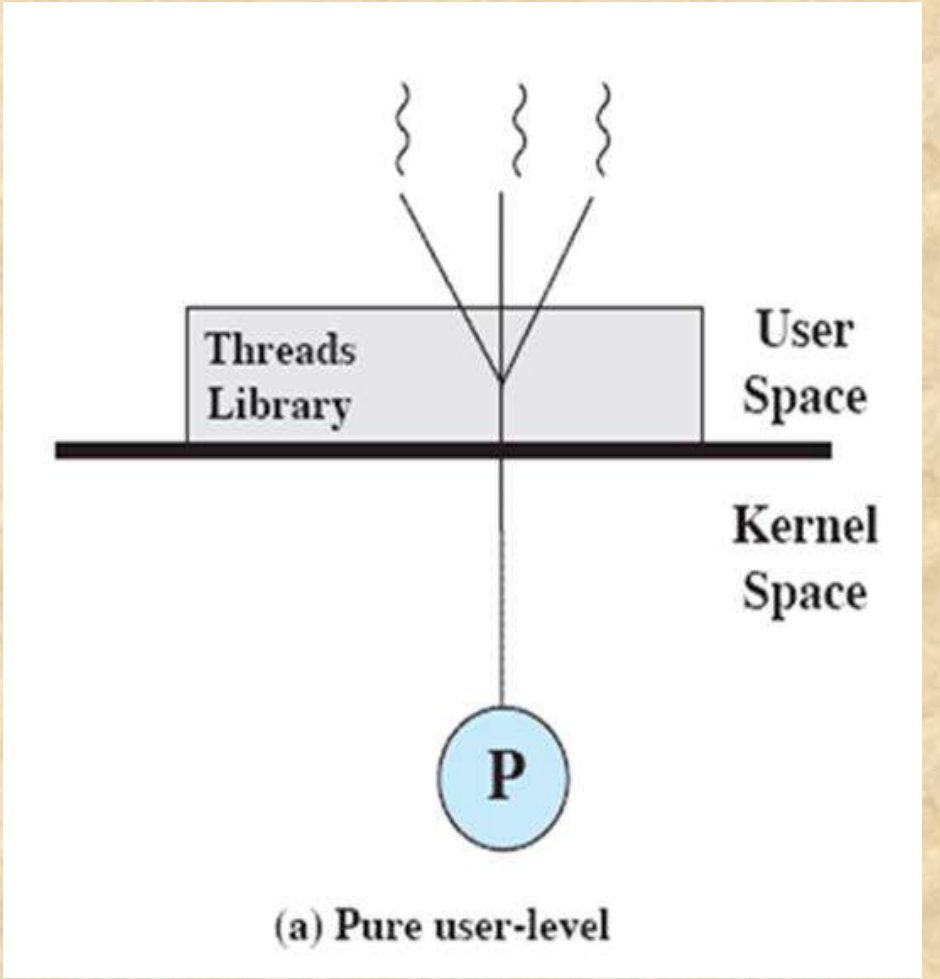
Kernel level Thread (KLT)

NOTE: we are talking about threads for *user* processes. Both ULT & KLT execute in user mode. An OS may also have threads but that is not what we are discussing here.



User-Level Threads (ULTs)

- Thread management is done by the application
- The kernel is not aware of the existence of threads



Relationships Between ULT States and Process States

Possible transitions from 4.6a:

4.6a→4.6b
4.6a→4.6c
4.6a→4.6d

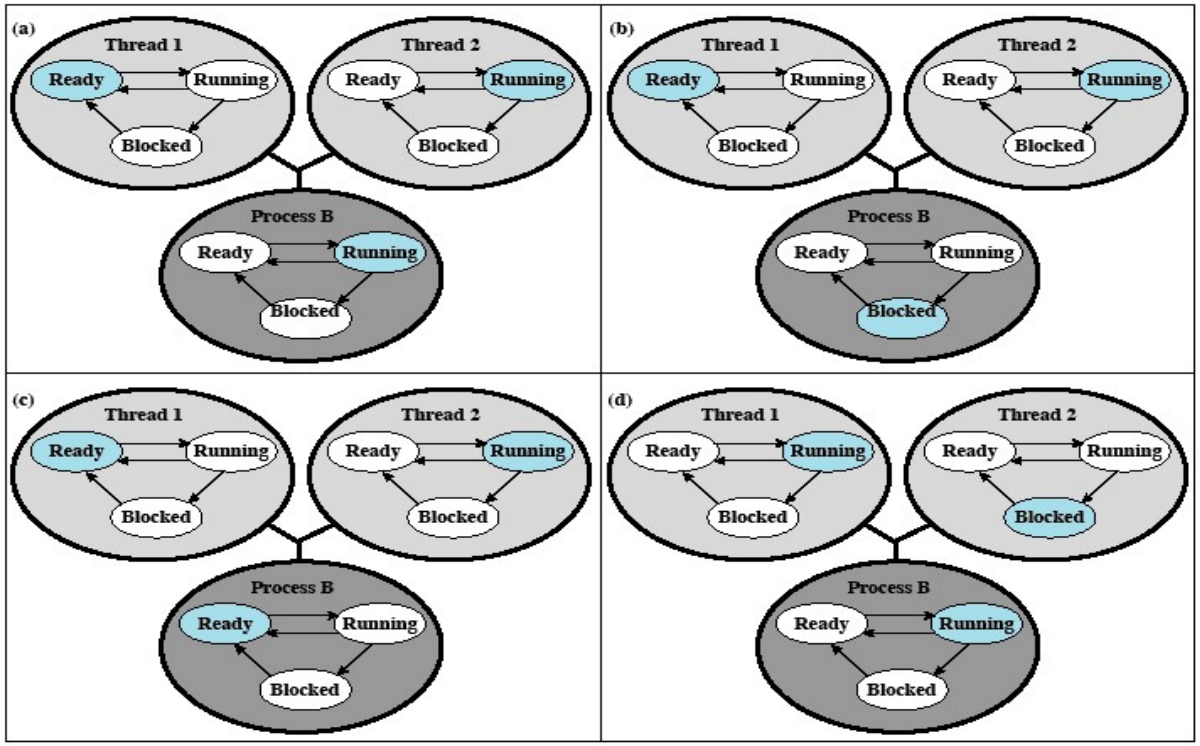
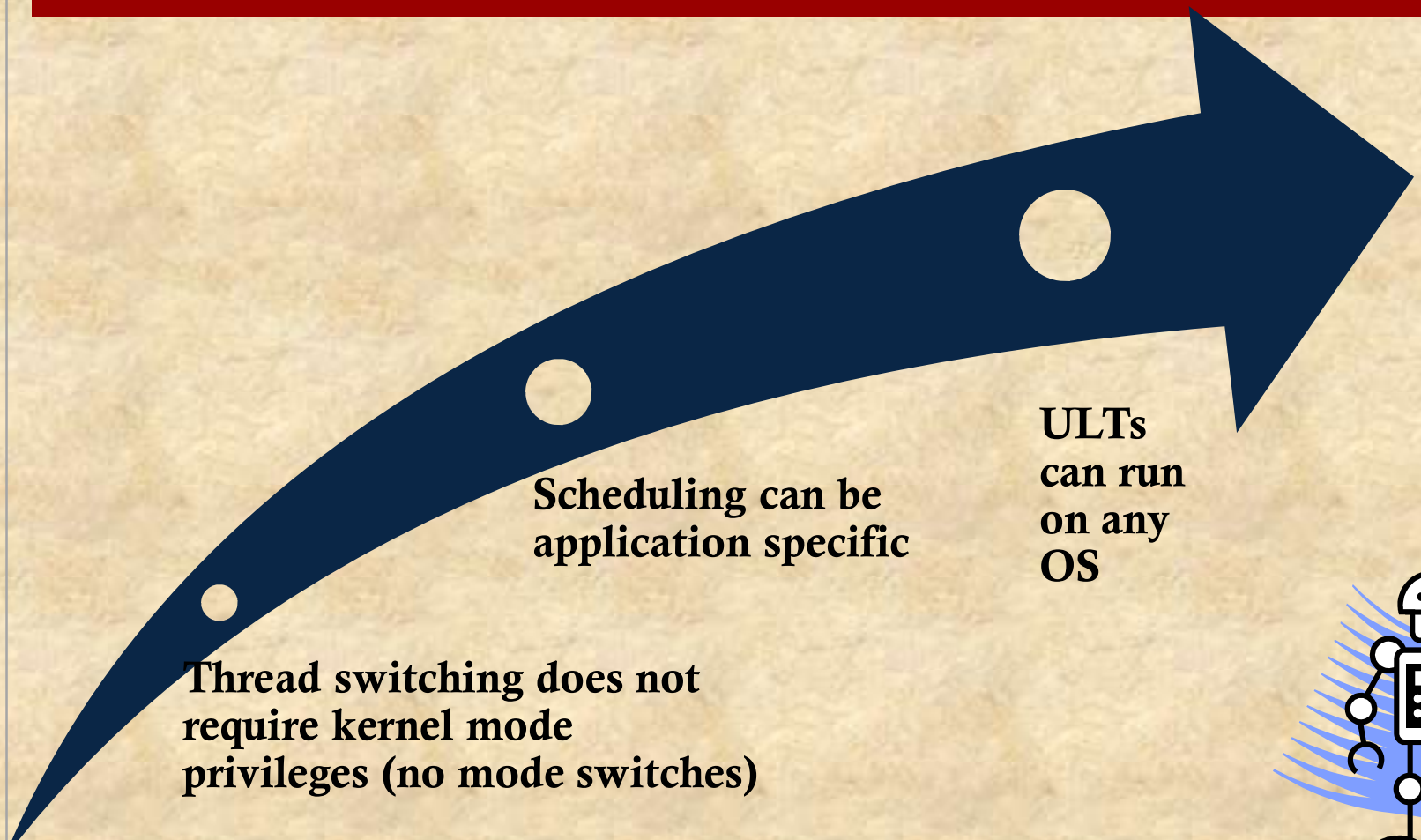


Figure 4.6 Examples of the Relationships between User-Level Thread States and Process States

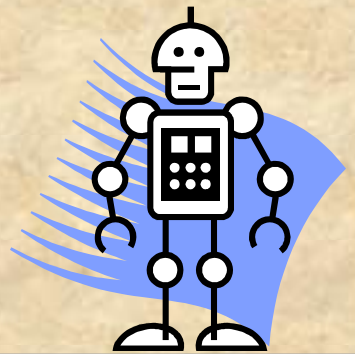
Advantages of ULTs



Thread switching does not require kernel mode privileges (no mode switches)

Scheduling can be application specific

ULTs can run on any OS



Disadvantages of ULTs

- In a typical OS many system calls are blocking
  - as a result, when a ULT executes a system call, not only is that thread blocked, but all of the threads within the process are blocked
- In a pure ULT strategy, a multithreaded application cannot take advantage of multiprocessing

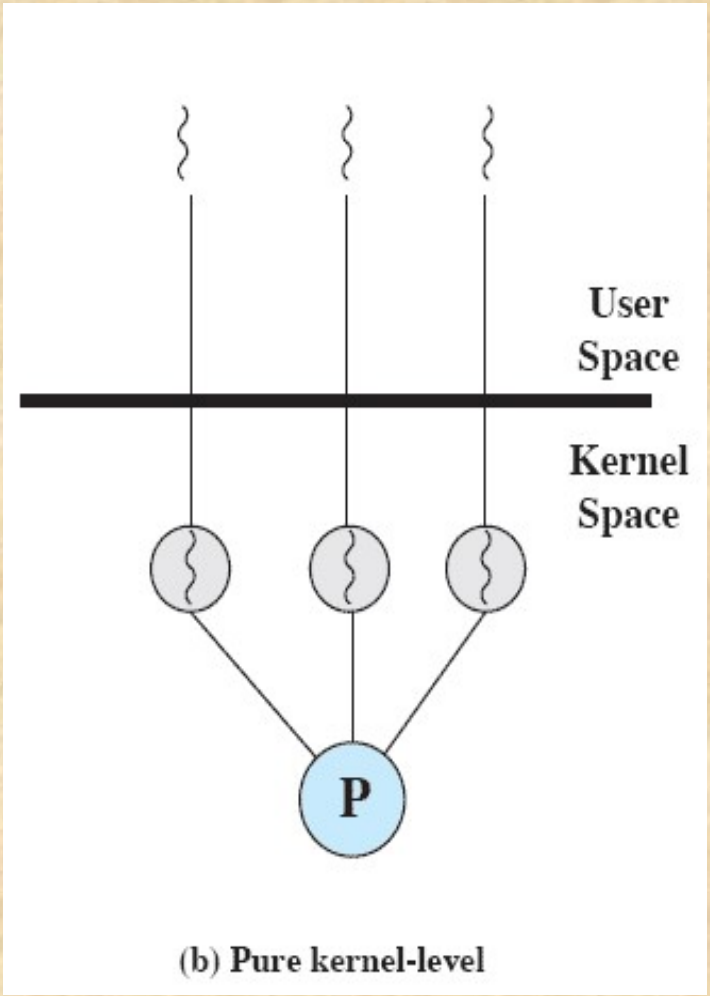
Overcoming ULT Disadvantages

Jacketing

- converts a blocking system call into a non-blocking system call

Writing an application as multiple processes rather than multiple threads

Kernel-Level Threads (KLTs)

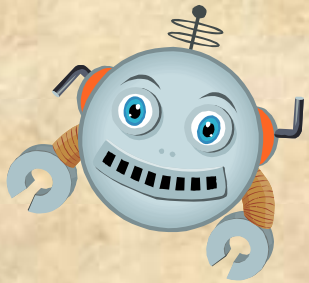


(b) Pure kernel-level

- Thread management is done by the kernel (could call them K*M*T)
  - no thread management is done by the application
  - Windows is an example of this approach

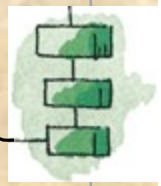
Advantages of KLTs

- The kernel can simultaneously schedule multiple threads from the same process on multiple processors
- If one thread in a process is blocked, the kernel can schedule another thread of the same process



Multicore Programming

- **Multicore** or **multiprocessor** systems putting pressure on programmers, challenges include:
  - **Dividing activities**
  - **Balance**
  - **Data splitting**
  - **Data dependency**
  - **Testing and debugging**
- ***Parallelism*** implies a system can perform more than one task simultaneously
- ***Concurrency*** supports more than one task making progress
  - Single processor / core, scheduler providing concurrency

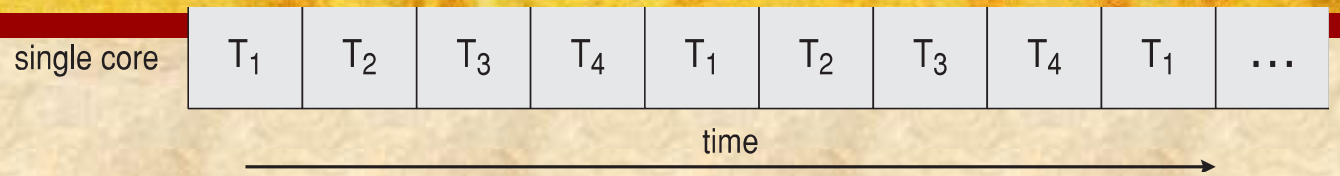


Multicore Programming (Cont.)

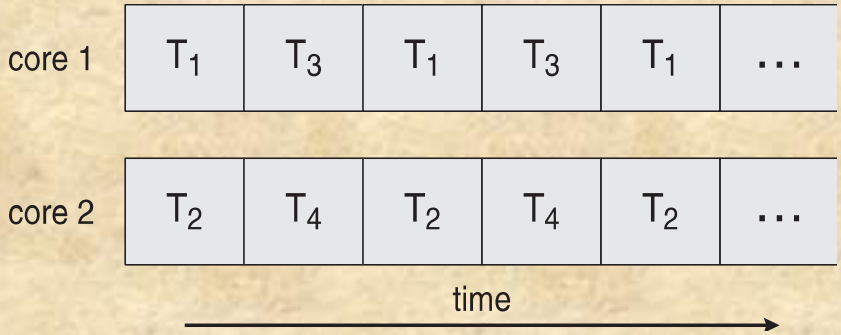
- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
  - **Task parallelism** – distributing threads across cores, each thread performing unique operation
- As # of threads grows, so does architectural support for threading
  - CPUs have cores as well as ***hardware threads***
  - Consider Oracle SPARC T4 with 8 cores, and 8 hardware threads per core

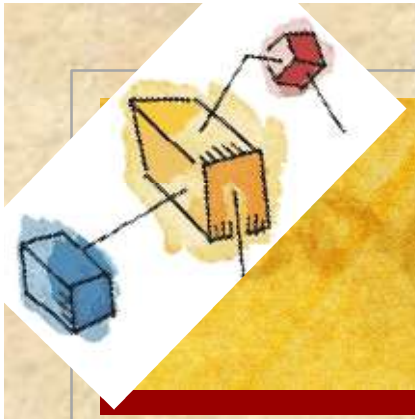
Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**



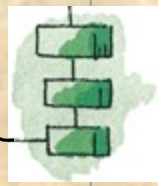
- **Parallelism on a multi-core system:**

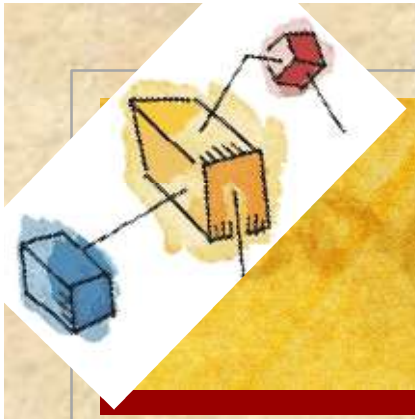




Multithreading Models

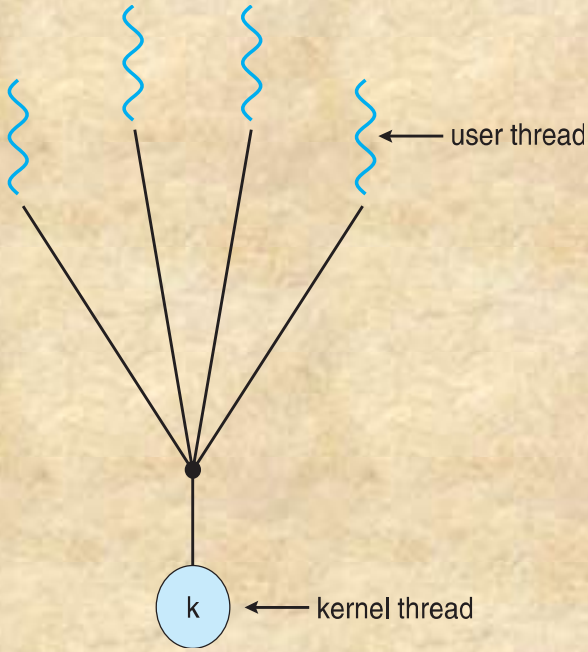
- Many-to-One
- One-to-One
- Many-to-Many

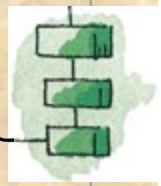


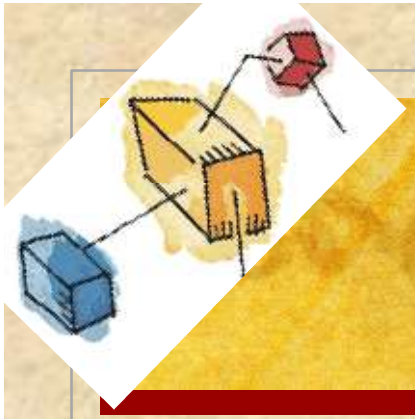


Many-to-One

- Many user-level threads mapped to single kernel thread
- One thread blocking causes all to block
- Multiple threads may not run in parallel on muticore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
  - **Solaris Green Threads**
  - **GNU Portable Threads**

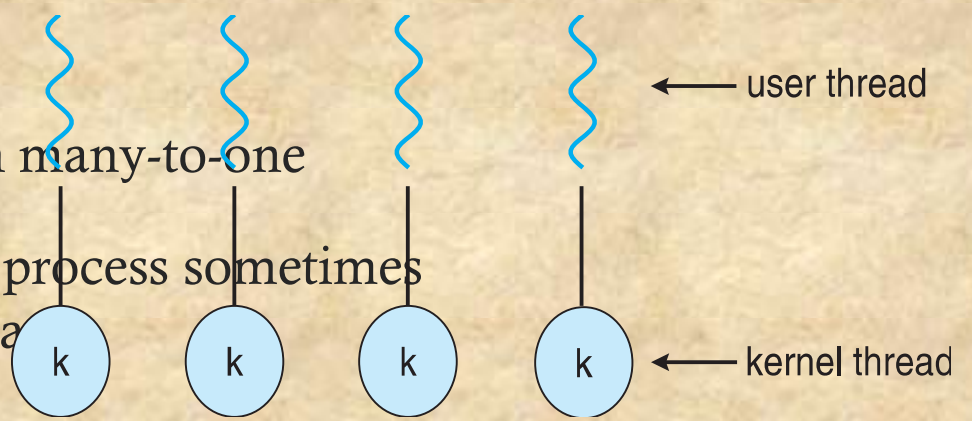


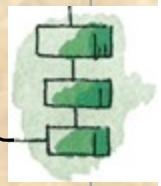


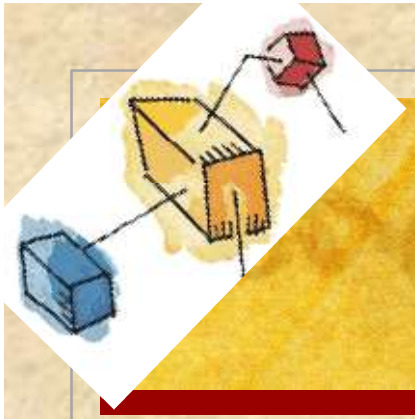


One-to-One

- Each user-level thread maps to kernel thread
- Creating a user-level thread creates a kernel thread
- More concurrency than many-to-one
- Number of threads per process sometimes restricted due to overhea
- Examples
  - Windows
  - Linux
  - Solaris 9 and later

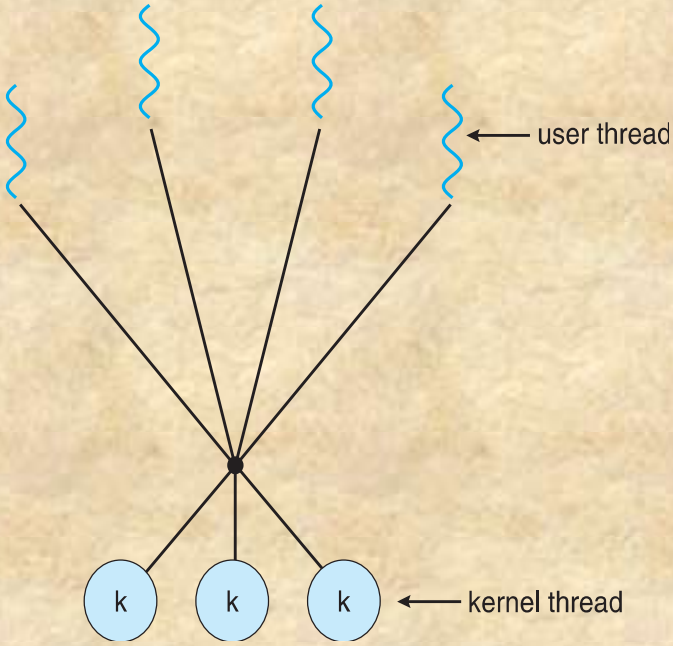


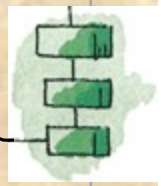


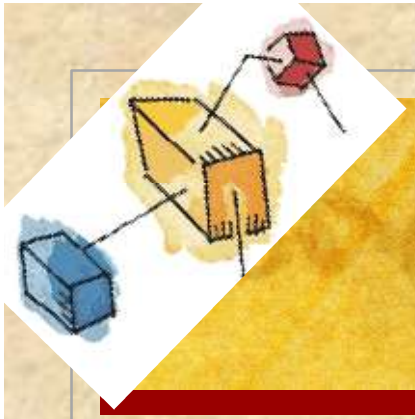


Many-to-Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Solaris prior to version 9
- Windows with the *ThreadFiber* package

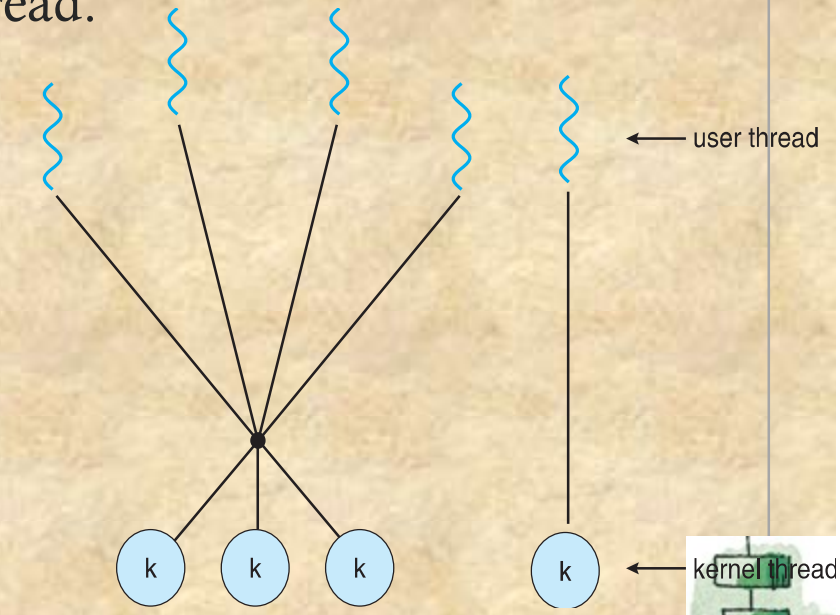






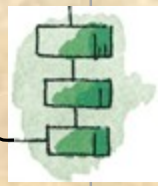
Two-level Model

- The two-level model is similar to the many-to-many model but also allows for certain user-level threads to be bound to a single kernel-level thread.
-



Implicit Threading

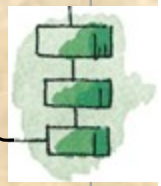
- Growing in popularity as numbers of threads increase, program correctness more difficult with explicit threads
- Creation and management of threads done by compilers and run-time libraries rather than programmers
- Three methods explored
  - Thread Pools
  - OpenMP
  - Grand Central Dispatch
- Other methods include Microsoft Threading Building Blocks (TBB), `java.util.concurrent` package

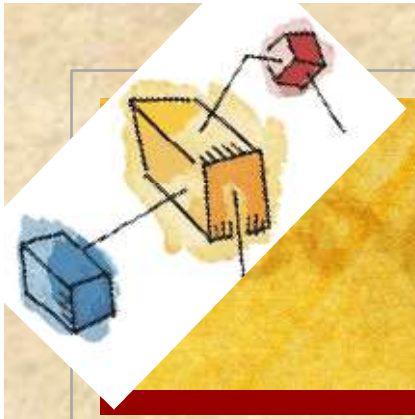


Thread Pools

- Create a number of threads in a pool where they await work
- Advantages:
  - Usually slightly faster to service a request with an existing thread than create a new thread
  - Allows the number of threads in the application(s) to be bound to the size of the pool
  - Separating task to be performed from mechanics of creating task allows different strategies for running task
- W

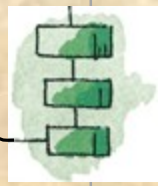
```
DWORD WINAPI PoolFunction(AVOID Param) {
    /*
    * this function runs as a separate thread.
    */
}
```

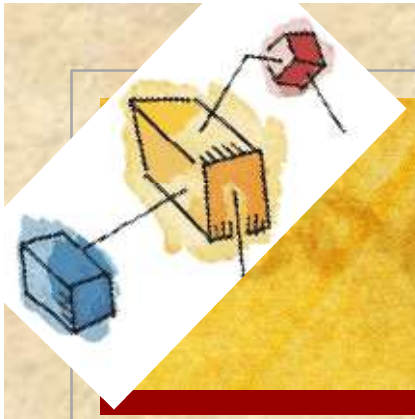




Threading Issues

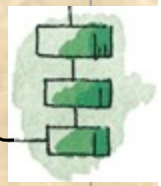
- Semantics of **fork()** and **exec()** system calls
- Signal handling
  - Synchronous and asynchronous
- Thread cancellation of target thread
  - Asynchronous or deferred
- Thread-local storage
- Scheduler Activations

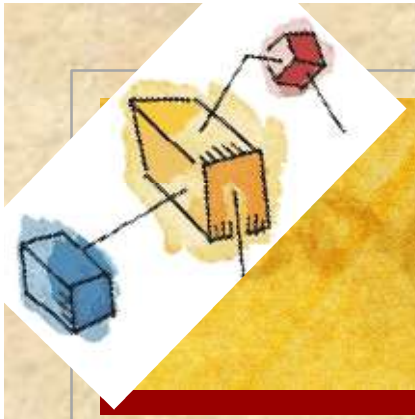




Semantics of fork() and exec()

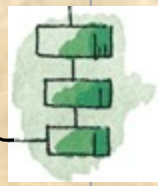
- Does **`fork()`** duplicate only the calling thread or all threads?
  - Some UNIXes have two versions of fork
- **`exec()`** usually works as normal – replace the running process including all threads





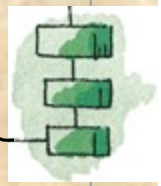
Signal Handling

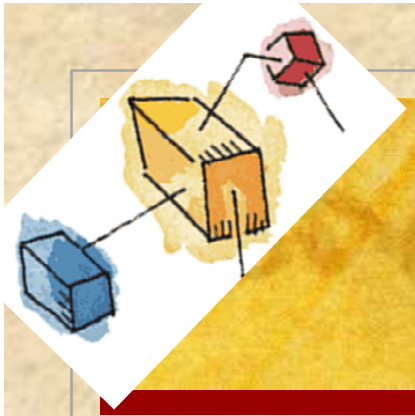
- **Signals** are used in UNIX systems to notify a process that a particular event has occurred.
- A **signal handler** is used to process signals
  1. Signal is generated by particular event
  2. Signal is delivered to a process
  3. Signal is handled by one of two signal handlers:
     1. default
     2. user-defined
- Every signal has **default handler** that kernel runs when handling signal
  - **User-defined signal handler** can override default
  - For single-threaded, signal delivered to process



Signal Handling (Cont.)

- Where should a signal be delivered for multi-threaded?
  - Deliver the signal to the thread to which the signal applies
  - Deliver the signal to every thread in the process
  - Deliver the signal to certain threads in the process
  - Assign a specific thread to receive all signals for the process





Thread Cancellation

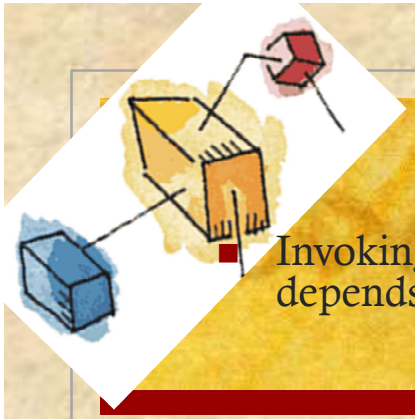
- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches:
  - **Asynchronous cancellation** terminates the target thread immediately
  - **Deferred cancellation** allows the target thread to periodically check if it should be cancelled
- Pthread code to create and cancel a threa

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

. . .

/* cancel the thread */
pthread_cancel(tid);
```

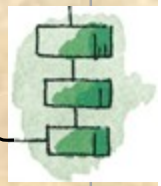


(Cont.)

- Invoking thread cancellation requests cancellation, but actual cancellation depends on thread state

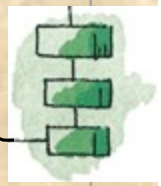
Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

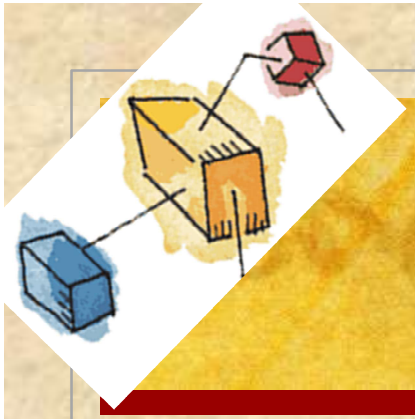
- If thread has cancellation disabled, cancellation remains pending until thread enables it
- Default type is deferred
  - Cancellation only occurs when thread reaches **cancellation point**
    - I.e. `pthread_testcancel()`
    - Then **cleanup handler** is invoked
- On Linux systems, thread cancellation is handled through signals



Thread-Local Storage

- **Thread-local storage** (**TLS**) allows each thread to have its own copy of data
- Useful when you do not have control over the thread creation process (i.e., when using a thread pool)
- Different from local variables
  - Local variables visible only during single function invocation
  - TLS visible across function invocations
- Similar to **`static`** data
  - TLS is unique to each thread





Scheduler Activations

- Both M:M and Two-level models require communication to maintain the appropriate number of kernel threads allocated to the application
- Typically use an intermediate data structure between user and kernel threads – **lightweight process** (**LWP**)
  - Appears to be a virtual processor on which process can schedule user thread to run
  - Each LWP attached to kernel thread
  - How many LWPs to create?
- Scheduler activations provide **upcalls** - a communication mechanism from the kernel to the **upcall handler** in the thread library
- This communication allows an application to maintain the correct number kernel threads